

UCCD3223 Mobile Applications Development

Mr. Tan Chiang Kang



History of Android



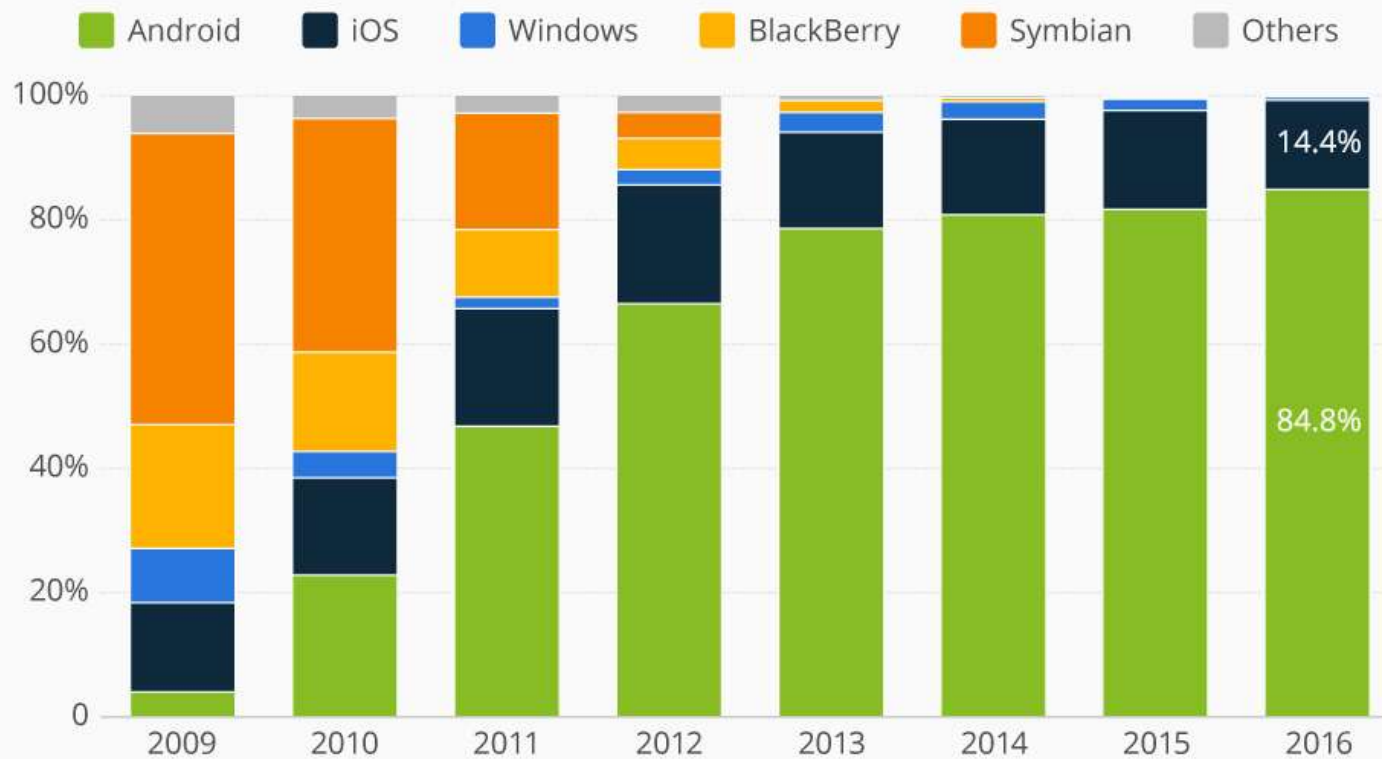
- Google acquired the startup company Android Inc. in 2005 to start the development of the Android Platform. The key players at Android Inc. included Andy Rubin, Rich Miner, Nick Sears, and Chris White.
- In late 2007, a group of industry leaders came together around the Android Platform to form the Open Handset Alliance.
- The Android SDK was first issued as an “early look” release in November 2007.
- In September 2008 T-Mobile announced the availability of HTC Dream (a.k.a. T-Mobile G1), the first smartphone based on the Android Platform.
- A few days after that, Google announced the availability of Android SDK Release Candidate 1.0.
- In October 2008, Google made the source code of the Android Platform available under Apache’s open source license.

Android vs iOS



Smartphone OS Market Share

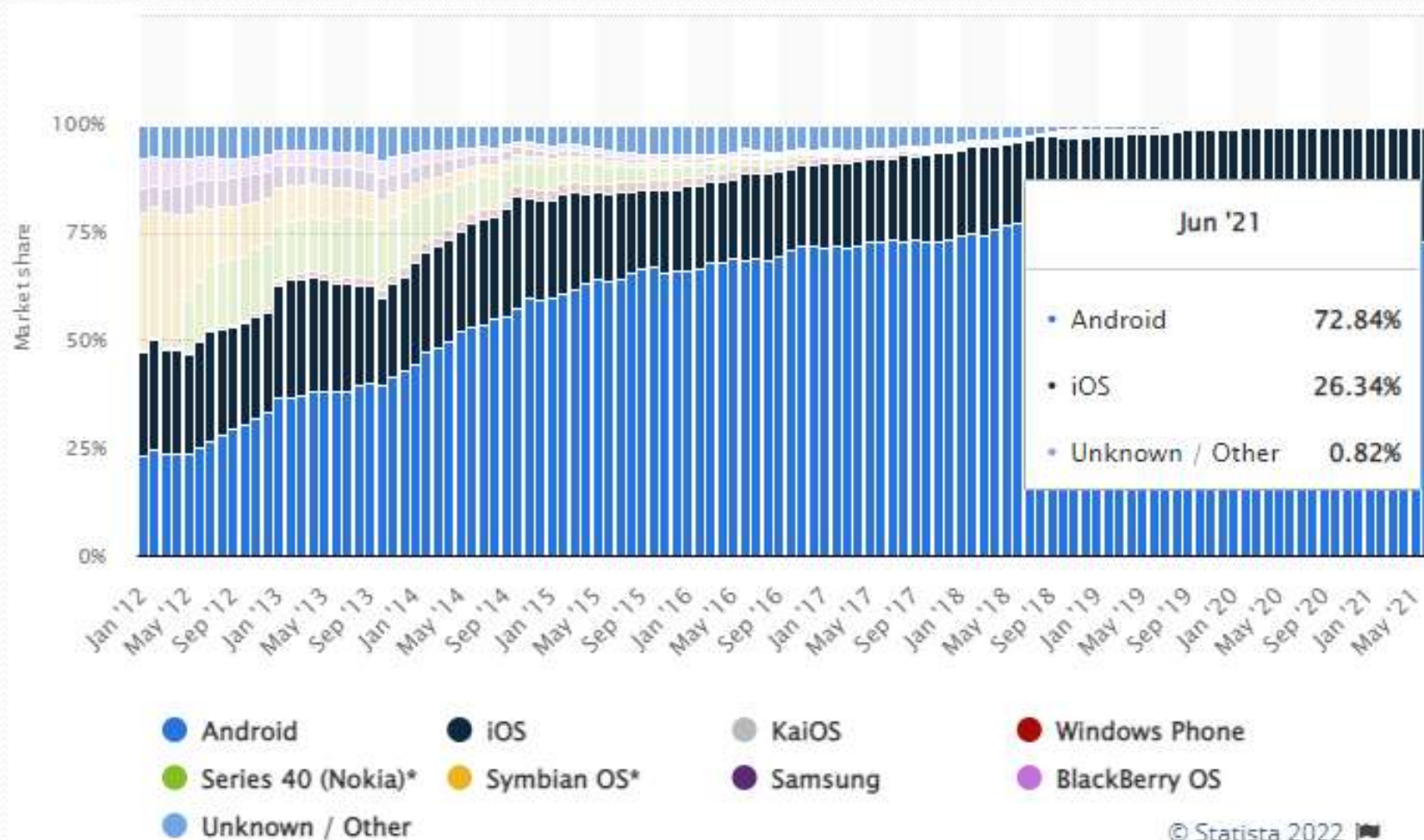
Worldwide smartphone operating system market share (based on unit sales)



@StatistaCharts Source: Gartner

statista

Smartphone OS Market Share



Other Mobile Platforms

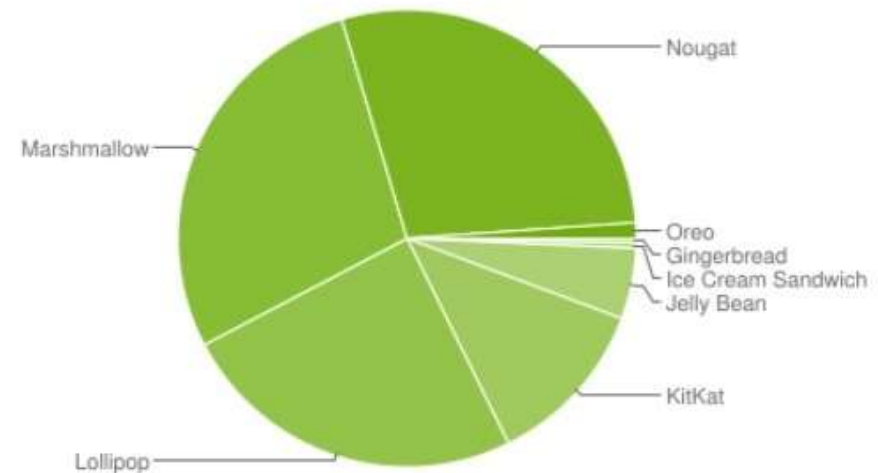
- Windows Mobile
- RIM Blackberry
- Symbian OS
- Samsung Bada
- webOS
- KaiOS
- Etc



Android Version Distribution

- Android versions progress very quickly ever since it was introduced with about 2 versions update released every year

Version	Codename	API	Distribution
2.3.3 - 2.3.7	Gingerbread	10	0.3%
4.0.3 - 4.0.4	Ice Cream Sandwich	15	0.4%
4.1.x	Jelly Bean	16	1.7%
4.2.x		17	2.6%
4.3		18	0.7%
4.4	KitKat	19	12.0%
5.0	Lollipop	21	5.4%
5.1		22	19.2%
6.0	Marshmallow	23	28.1%
7.0	Nougat	24	22.3%
7.1		25	6.2%
8.0	Oreo	26	0.8%
8.1		27	0.3%



Data: Feb 2018

Taken from androidpolice.com

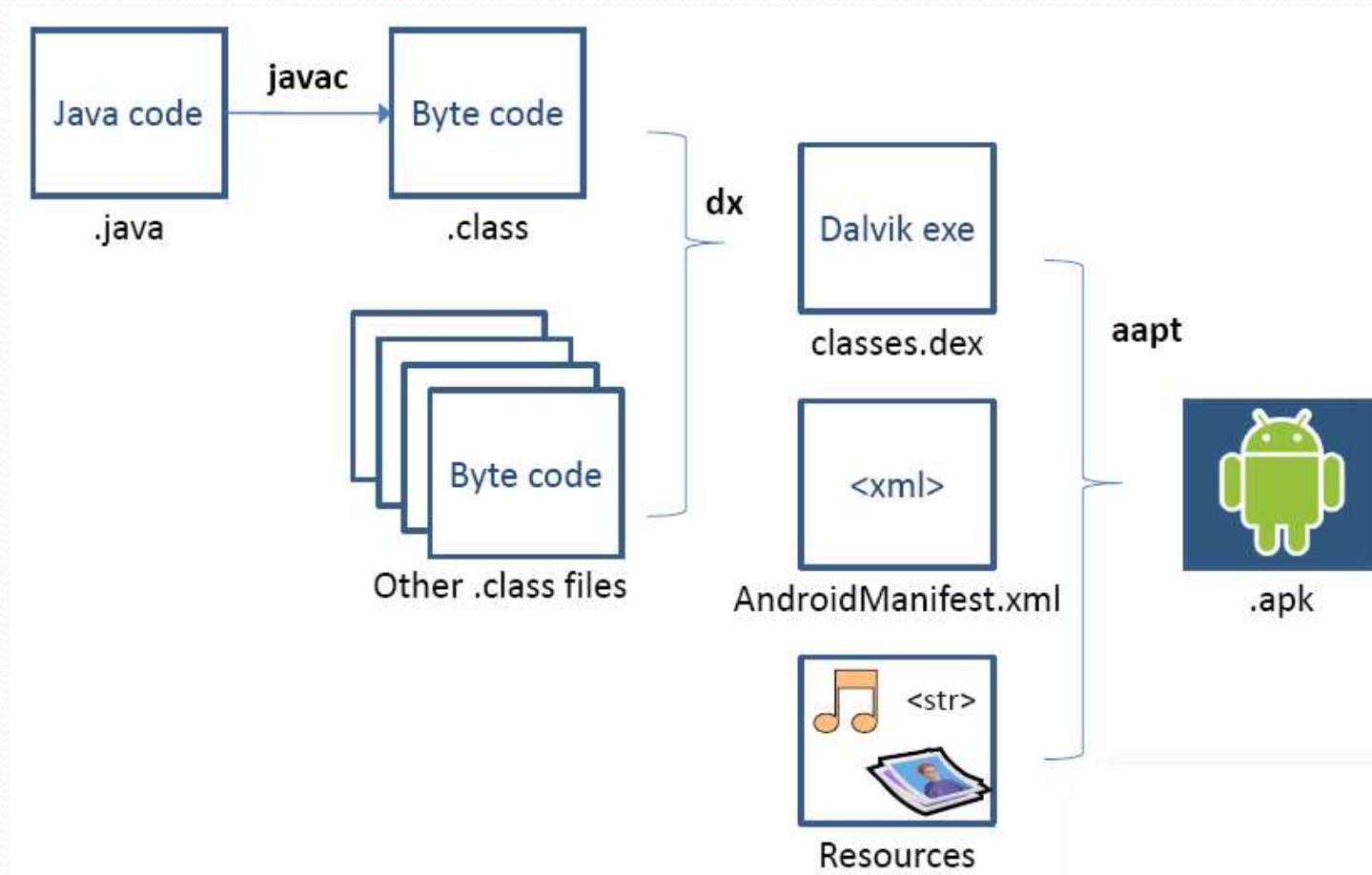
What can Android do?

- Storage
 - databases or files
- Network
 - Android devices will generally be Internet-ready
 - Raw Java sockets available
 - *Built-in WebKit-based Web browser (as Safari) widget you can embed in your application

What can Android do?

- Multimedia
 - play back and record audio and video
 - capture image and video
- GPS
 - access to location information
- Phone and messaging services
 - initiate phone calls
 - send and receive SMS messages

Concept of Android Apps



Android Runtime

- Dalvik VM
 - Dex files
 - Compact and efficient than class files
 - Limited memory and battery power
- ART (Replacing Dalvik VM)
 - Ahead-of-time (AOT) compilation
 - Default for Android 5 (API 21) and higher
- Core Libraries
 - Java 5 Std edition
 - Collections, I/O etc...



Four Application Components

- **Activities**
 - Presents a visual UI for a single endeavor
 - Examples: list of photos, buttons to start/stop a song
- **Services**
 - Performs background work (no UI)
 - Examples: play background music, retrieve data over a network
- **Broadcast Receivers**
 - Receives and reacts to broadcast announcements (no UI)
 - Broadcast examples: battery is low, picture is taken, language preferences changed
- **Content Providers**
 - Provides app data to other applications (no UI)
 - Examples: share contact info from SQLite, provide image from the file system

Activities

- An activity is a window that contains the UI of your applications.
- Typically, applications have one or more activities.
- The main aim of an activity is to interact with the user.
- From the moment an activity appears on your screen, to the moment it is hidden, it goes through a number of stages.
- Those stages are known as your application's life cycle.
- Understanding the life cycle of an activity is important to ensure the application you are building works correctly.

Activities: Example

- Lets create a Java class that extends the activity base class. Your main Java file should contain the following:

```
package my.edu.utar.example;
```

```
import android.app.Activity;  
import android.os.Bundle;
```

```
public class LifecycleActivity extends Activity {  
    /** Called when the activity is first created. */  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.main);  
    }  
}
```

Activating Components

- Content providers are activated by a ContentResolver
- Activities, Services, & Broadcast Receivers are activated by **Intent**
- Intents are asynchronous messages
- Example: A Service starts an Activity to pick a photo by using an Intent. The photo is returned to the Service also using an Intent.

Manifest Overview

- Every Android app project must have an `AndroidManifest.xml` file at the root of the project source set.
- The manifest file describes essential information about your app to the Android build tools, the Android operating system, and Google Play.
- It is bundled into Android package (.apk file).

Manifest Overview (cont'd)

- The manifest file is required to declare the following:
 - The **app's package** name which usually matches your code's namespace. The Android build tools use this to determine the location of code entities when building your project. When packaging the app, the build tools replace this value with the application ID from the Gradle build files, which is used as the unique app identifier on the system .

Manifest Overview (cont'd)

- The manifest file is required to declare the following:
 - The **components of the app** which include all Activities, Services, Broadcast Receivers, and Content Providers. It can also declare capabilities such as which device configurations it can handle, and intent filters that describe how the component can be started.
 - The **permissions** that the app needs in order to access protected parts of the system or other apps. It also declares any permissions that other apps must have if they want to access content from this app

AndroidManifest.xml

- This is how an *AndroidManifest.xml* file looks like:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="org.nav6.lifecycle">

    <application android:icon="@drawable/icon" android:label="@string/app_name">
        <activity android:name=".LifecycleActivity"
            android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

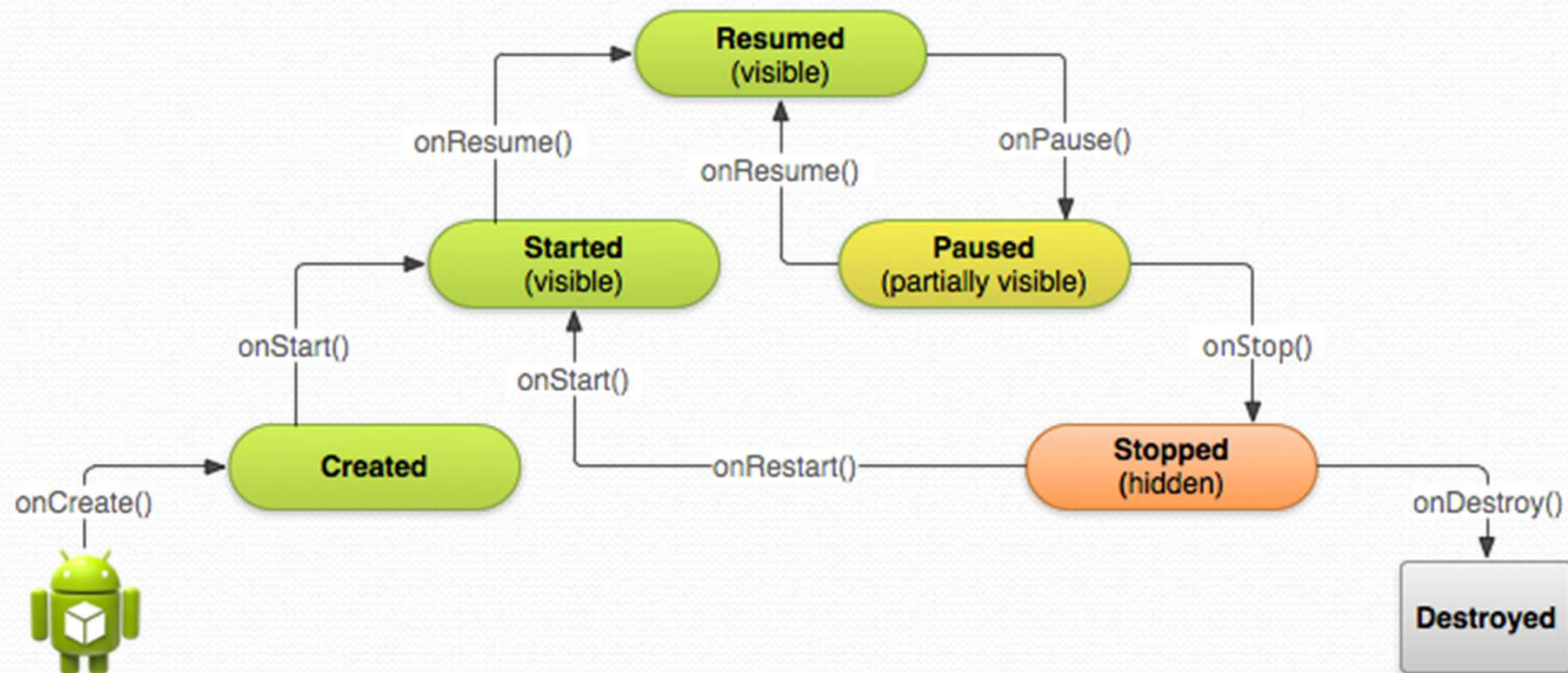
The Life Cycle

- Mobile applications are different compared to desktop applications
 - Actions are mostly not started from a main function
 - They evolve according to the life cycle of the application
 - The programmer can program an action that should be done during a stage of the life cycle, it will do something when such stage happens or was being triggered
 - Predefined abstract functions were usually available and can be overridden

Life Cycle of Activity (Android)

- The Activity base class defines a series of events that governs the life cycle of an activity. The Activity class defines the following events:
 - ***onCreate()*** – Called when the activity is first started.
 - ***onStart()*** – Called when the activity becomes visible to the user
 - ***onResume()*** – The activity is in the foreground and user can interact with it
 - ***onPause()*** – Called when another activity comes to the foreground and the activity is partially obscured.
 - ***onStop()*** – The activity is completely hidden and no longer visible to the user.
 - ***onRestart()*** – From the Stopped state, the activity either comes back to interact with the user or the activity finishes and goes away. If the activity comes back, the system invokes ***onRestart()***.
 - ***onDestroy()*** – Activity is destroyed and removed from the memory.

Life Cycle of Activity (Android)



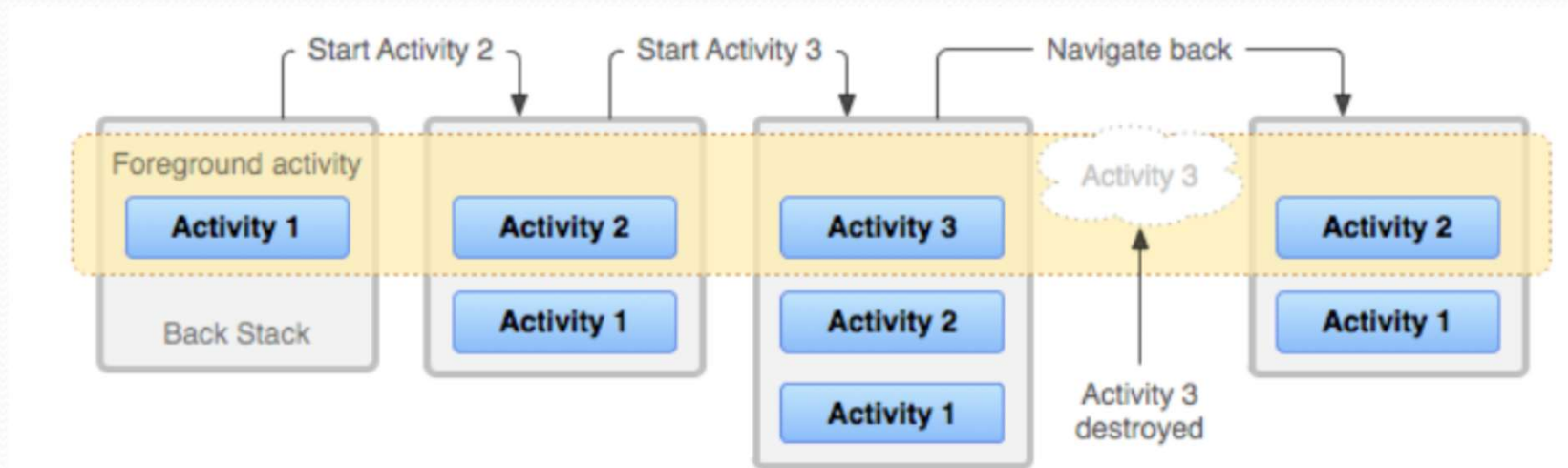
Task and Back Stack

- The Activities are arranged in a stack – the back stack – in the order in which each Activity is opened.
- For example, an **Email App** might have one Activity to show a list of new messages. When the user selects a message, a new Activity opens to view that message. This new Activity is added to the back stack. If the user presses the **Back** button, that new activity is finished and popped off the stack.
- When the current Activity starts another, the new Activity is pushed on the top of the stack and takes focus.

Task and Back Stack (cont'd)

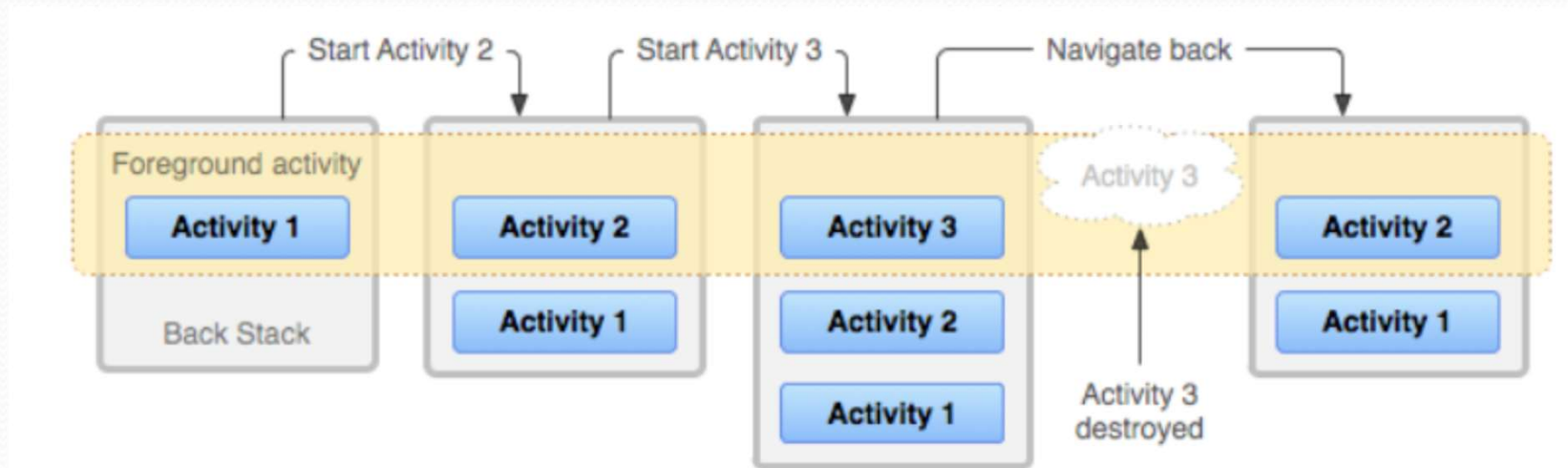
- The previous Activity remains in the stack, but is stopped. When an Activity stops, the system retains the state of its user interface.
- When the user presses the **Back** button, the current Activity is popped from the top of the stack (the Activity is destroyed) and the previous Activity resumes (the previous state of its UI is restored).
- Activities in the stack are never rearranged, only pushed and popped from the stack.

Task and Back Stack (cont'd)



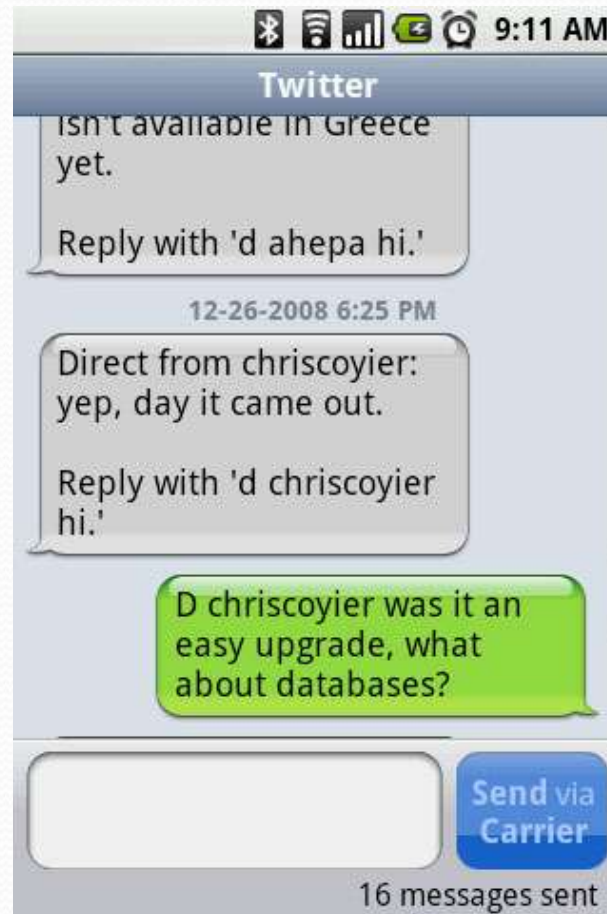
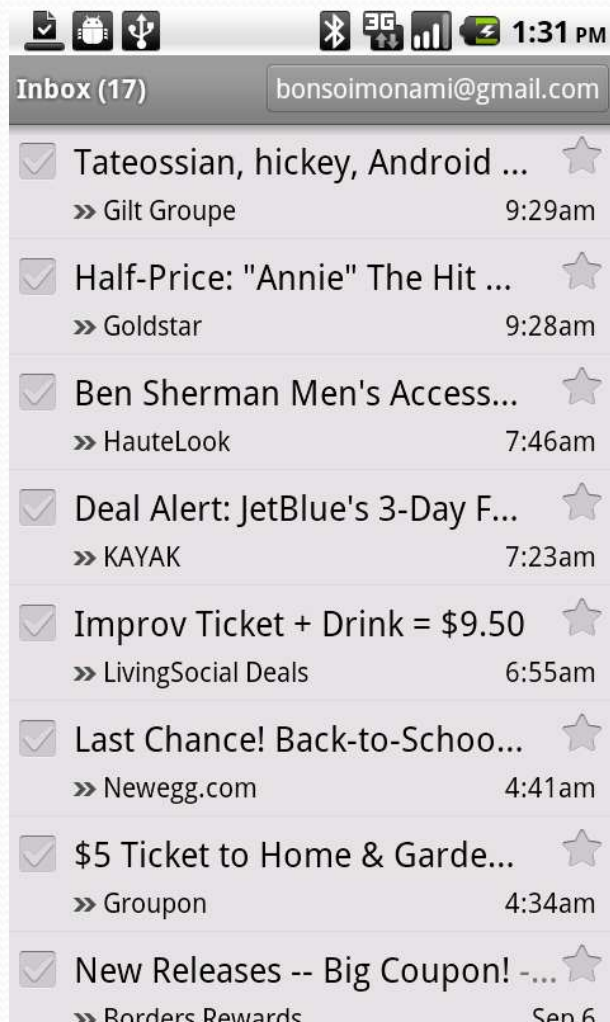
- Each new Activity in a task adds an item to the back stack. When the user presses the **Back** button, the current Activity is destroyed and the previous Activity resumes.

Task and Back Stack (cont'd)



- If the user continues to press **Back**, then each Activity in the stack is popped off to reveal the previous one, until the user returns to the Home screen (or to whichever Activity was running when the task began). When all Activities are removed from the stack, the task no longer exists.

Example of Android Apps



MAD Considerations

- Strategic rationale
 - Business context - The rationale of implementing a mobile app solution, whether it is for B2E internal workers, B2C customers, or B2B clients
- Architecture
 - Architecture options - Evaluating Wrapper/Hybrid (Ionic), Native Mappers (React Native), Canvas Engines (Flutter) or Native iOS/Android
- Client-Side Resources & Capabilities
 - Mobile resources - Physical hardware restrictions and capabilities
 - User Interface (UI) - Ergonomics, accessibility features, responsiveness across various aspect ratios
 - New device APIs & sensors – On-device AI processing, biometric cryptography Passkey, location-based sensors (UWB/BLE) etc

MAD Considerations

- Data Layer, Infrastructure & Connectivity
 - Network infrastructure – connectivity profile (5g, Wi-Fi, low bandwidth environments)
 - Data transfer & protocols – Client-to-server patterns, peer-to-peer data syncing, REST APIs, or real-time WebSockets
 - Legacy systems – Integration hooks into existing corporate databases, SOAP web services
 - Offline strategy – local vs. remote, conflict resolution when reconnection occurs?
- Security & Compliance
 - Security requirements – User authentication (OAuth 2.0, Multi-Factor etc), data-at-rest encryption, data-in-transit encryption, and run-time protection
 - Privacy & regulatory compliance – Compliance with local and international data acts

MAD Considerations

- Development Management
 - In-house vs. outsourced development and long-term SLA/technical support
 - Separate iOS/Android teams vs. a unified cross-platform team
 - Agile/scrum development methodology?
- Lifecycle Sustainability
 - Annual OS adaptation
 - Dependency & patch management
 - App store compliance operations

How to start development?

- To simply develop a mobile application, the developer can first identify the following:
 - **Target platform:** The developer to figure out which of the platform(s) among the available choices are selected as the target deployment platform(s)
 - **Tools:** Depending on the targeted platform(s), there are sometimes a few different tools that can be chosen for the development

Cross-platform

- Not all mobile developments are done specifically for an OS, some development languages offer cross-platform capabilities
- Cross-platform technologies allowed developers to develop once and adapt to multiple platforms quickly
- Some of the approaches are:
 - making use of virtual machines
 - using a wrapper framework over the application
 - cross compiling to run natively on different platforms (including UI)

Cross-platform

- Java ME is one good example of such capability but has aged and technically given up by the developers
- Other examples of cross-platform development are:
 - Kotlin Multiplatform
 - Flutter
 - React Native
 - Unity
 - Ionic
 - .NET MAUI
 - Etc



Cross-platform

- Cross-platform via wrapper framework can be achieved through web applications, where they are interpreted using a web browser on different platforms
 - Such hybrid applications can share the same code base as web applications, but instead are wrapped in a native container to run on different platforms
- In any case, cross-platform applications often do not come with the best performance and feature set compared to native applications, due to additional abstraction and lack of access to the latest native libraries or hardware

What will you learn in MAD?

- 3 basic parts in mobile applications development:
 - **User Interface** – How to design a mobile application UI? What to display? How to interact with it?
 - **In-phone Storage** – What do we need to store in the phone? How can we store something in the phone?
 - **Networking** – When do we need to connect to the Internet? What can we do for the mobile app by connecting to the Internet?

User Interface Design

- Graphical User Interface (GUI) is commonly available for mobile applications nowadays
- The design of the interface that you need to take into consideration of:
 - Easy to see and read
 - Easy to navigate
 - Easy to input

User Interface Design

- The designs are based on physical limitations and different features of a mobile device:
 - Screen size - considerably much smaller compared to a proper PC screen
 - Input methods - mainly touchscreen-based today, a very different input experience compared to a traditional PC/laptop, especially when it comes to typing. Gesture-based is possible with touchscreen input
 - Computation power - UI should not be too computationally expensive to cater for smaller processor sizes

User Interface Design

- Therefore, the outcome of design are often:
 - Simpler design compared to their PC counterpart, often focusing on the most crucial features
 - Reduction in necessity to type
 - Dropdown menu, auto-complete, gesture input etc
 - Output display aware
 - Elements to be displayed optimised accordingly
 - Smoother user experience
 - Facilitated by in-phone storage (of app states), and other hardware (e.g. fingerprint sensor, camera for face recognition etc)

In-Phone Storage

- We may want to store some data permanently in the phone
- The general type of data that can be saved:
 - Simple data pairs (preferences, settings, etc)
 - Database (tabled data, more complex)
 - Files
 - Specific type of data (contact list, calendar, etc)
- Can data be stored elsewhere?

Networking

- Today, Internet connectivity is a basic requirement expected from smartphones
 - Connection can be via different means, e.g. cellular network, and Wi-Fi through fibre broadband etc
- The network connection allows us to go online, and hence able to browse websites using browsers installed on the mobile devices
- What else can we do by connecting to the Internet?

Networking

- Other than webpages, the Internet also host web services:
 - allowing applications to access data stored on your own (cloud) servers and/or other services
 - allowing applications to access data stored made available by third parties and/or other services

What's Next?

- High Level User Interface